

COMP201

Computer Systems & Programming

Lecture #24 – Wrapping Up



KOÇ
UNIVERSITY

Aykut Erdem // Koç University // Fall 2025

Plan for Today

- **Recap:** Where We've Been
- COMP201 Tools and Techniques
- What's Next?
- Q&A

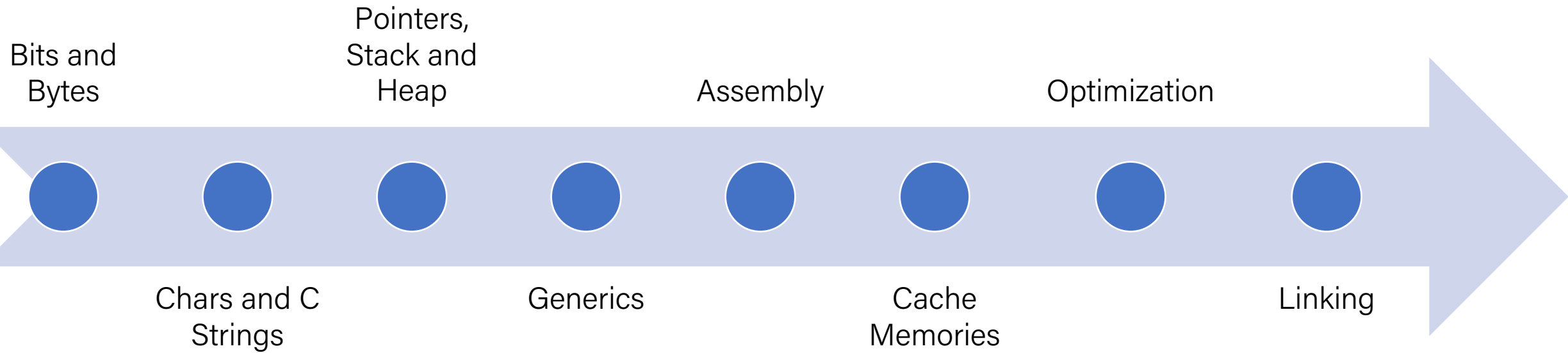
Disclaimer: Slides for this lecture were borrowed from
—Nick Troccoli's Stanford CS107 class

Lecture Plan

- **Recap:** Where We've Been
- COMP201 Tools and Techniques
- What's Next?
- Q&A

We've covered *a lot* in just
14 weeks! Let's take a look
back.

Our COMP201 Journey



Course Overview

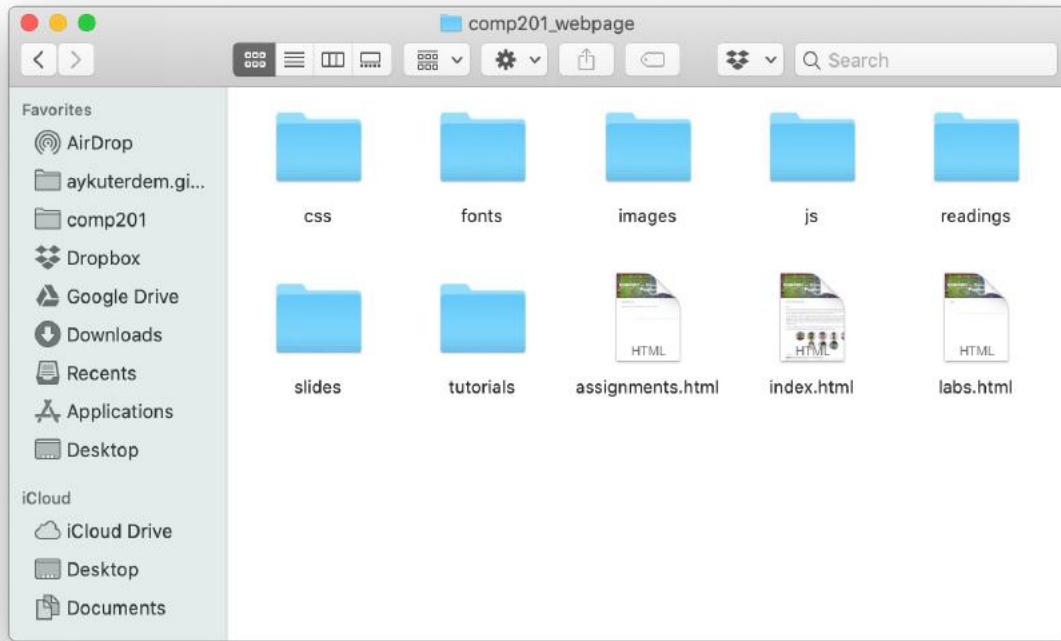
1. **Bits and Bytes** - *How can a computer represent integer and float numbers?*
2. **Chars and C-Strings** - *How can a computer represent and manipulate more complex data like text?*
3. **Pointers, Stack and Heap** – *How can we effectively manage all types of memory in our programs?*
4. **Generics** - *How can we use our knowledge of memory and data representation to write code that works with any data type?*
5. **Assembly** - *How does a computer interpret and execute C programs?*
6. **Cache Memories** - *How does the memory system is organized as a hierarchy of different storage devices with unique capacities?*
7. **Optimization**- *How we can optimize our code to improve efficiency and speed? The optimizations GCC can perform.*
8. **Linking**- *How to construct programs from multiple object files?*

First Day

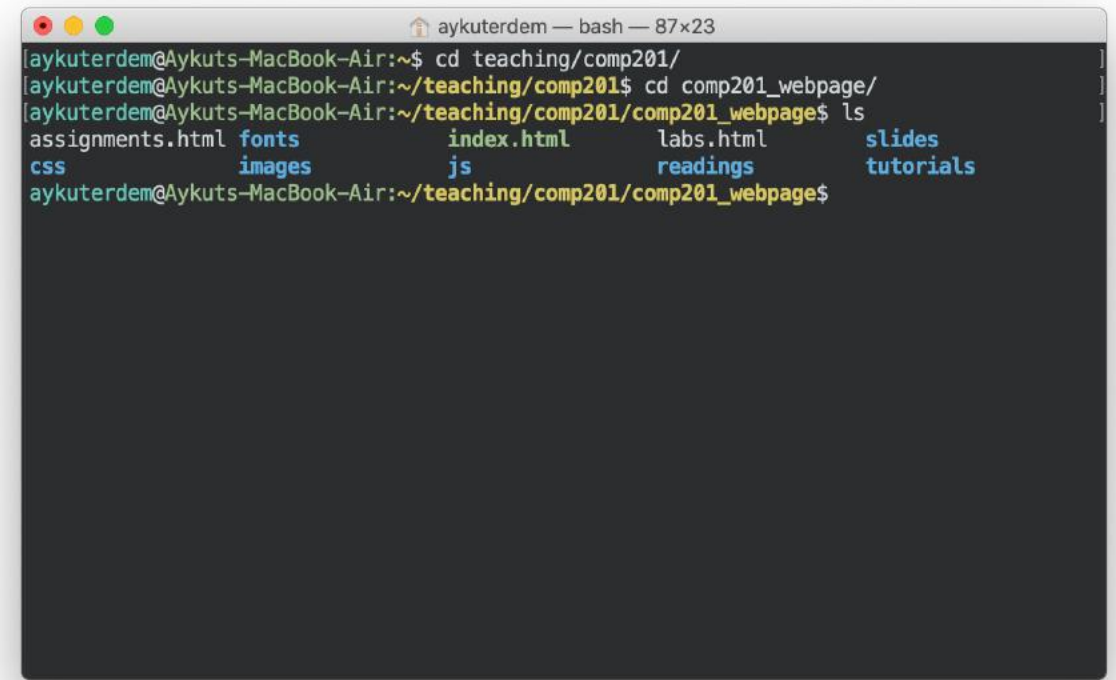
```
/*  
 * hello.c  
 * This program prints a welcome message  
 * to the user.  
 */  
#include <stdio.h>    // for printf  
  
int main(int argc, char *argv[]) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

First Day

- The **command-line** is a text-based interface to navigate a computer, instead of a Graphical User Interface (GUI).



Graphical User Interface

A screenshot of a terminal window titled 'aykuterdem — bash — 87x23'. The terminal shows the following commands and output:

```
aykuterdem@Aykuts-MacBook-Air:~$ cd teaching/comp201/  
aykuterdem@Aykuts-MacBook-Air:~/teaching/comp201$ cd comp201_webpage/  
aykuterdem@Aykuts-MacBook-Air:~/teaching/comp201/comp201_webpage$ ls  
assignments.html  fonts          index.html     labs.html      slides  
css              images        js             readings       tutorials  
aykuterdem@Aykuts-MacBook-Air:~/teaching/comp201/comp201_webpage$
```

Text-based interface

What is your favorite programming language?



Python



36%

Java



52%

C++



4%

C



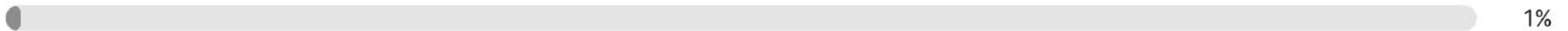
6%

R



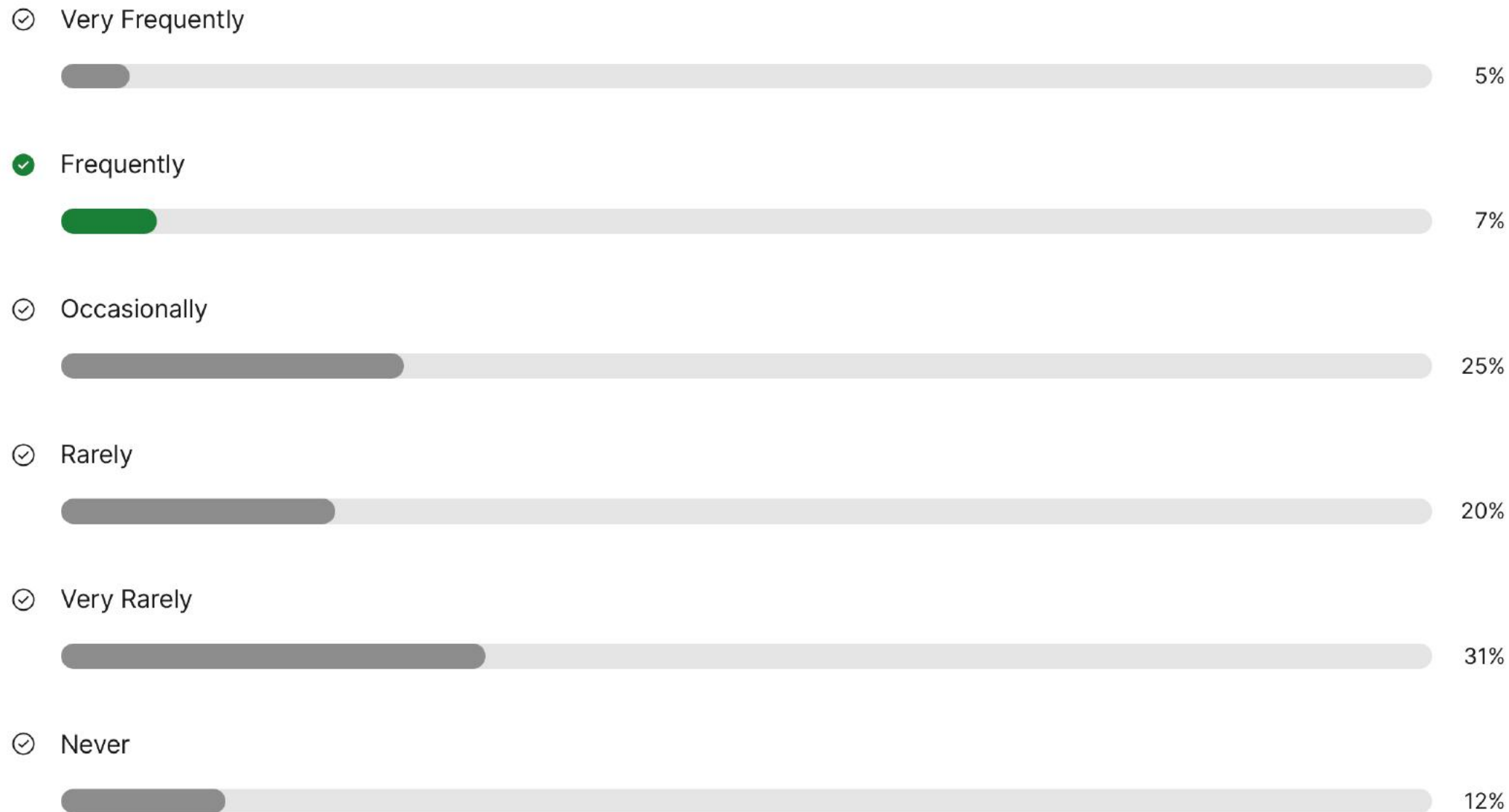
0%

Other



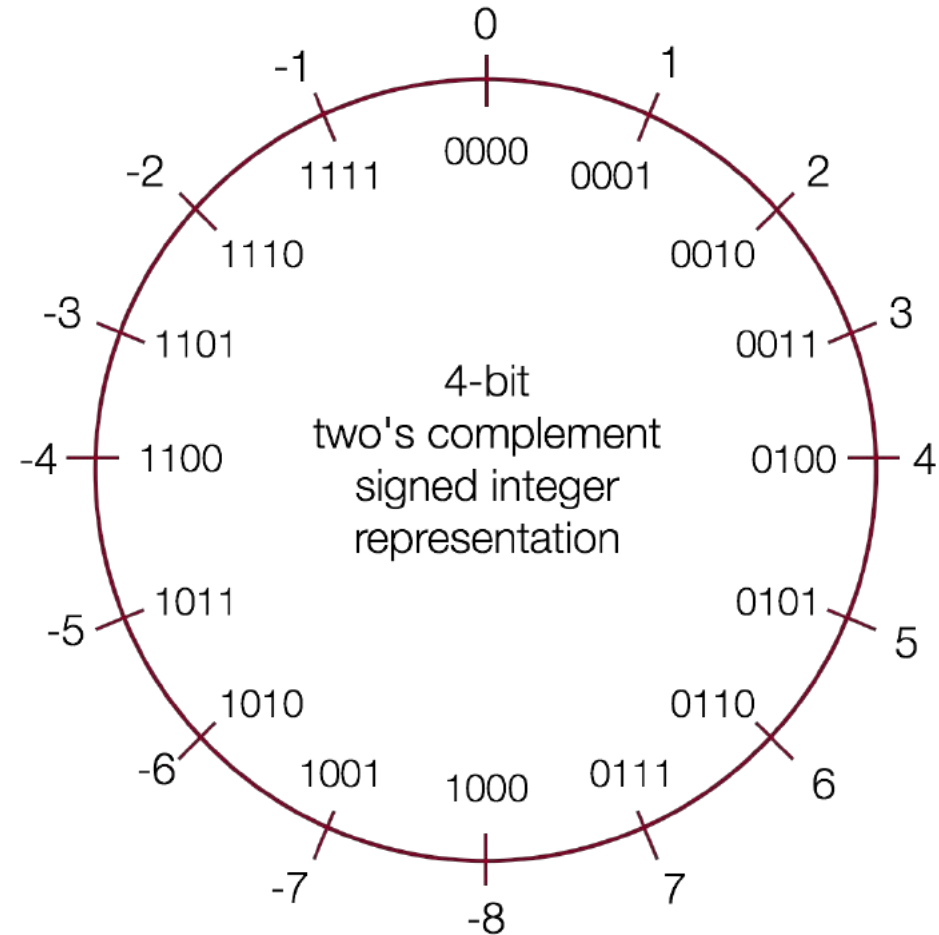
1%

How often do you use the command line (i.e. Mac OS X Terminal, Linux shell or may be Windows PowerShell)?



Bits And Bytes

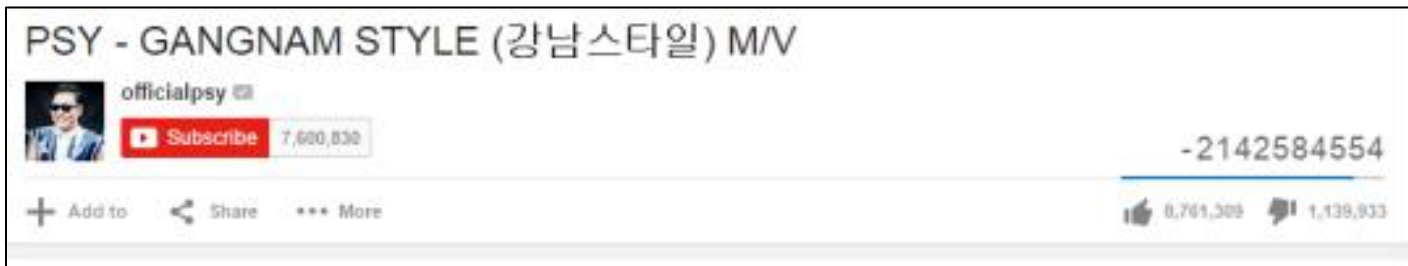
Key Question: *How can a computer represent integer numbers?*



Bits And Bytes

Why does this matter?

- Limitations of representation and arithmetic impact programs!
- We can also efficiently manipulate data using bits.



<https://kotaku.com/why-gandhi-is-such-an-asshole-in-civilization-1653818245>

Floats

- IEEE Floating Point is a carefully-thought-out standard. It's complicated, but engineered for their goals.
- Floats have an extremely wide range, but cannot represent every number in that range.
- Some approximation and rounding may occur! This means you definitely don't want to use floats e.g. for currency.
- Associativity does not hold for numbers far apart in the range
- Equality comparison operations are often unwise.

C Strings

Key Question: *How can a computer represent and manipulate more complex data like text?*

- Strings in C are arrays of characters ending with a null terminator!
- We can manipulate them using pointers and C library functions (many of which you could probably implement).

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
value	'H'	'e'	'l'	'l'	'o'	','	' '	'w'	'o'	'r'	'l'	'd'	'!'	'\0'

C Strings

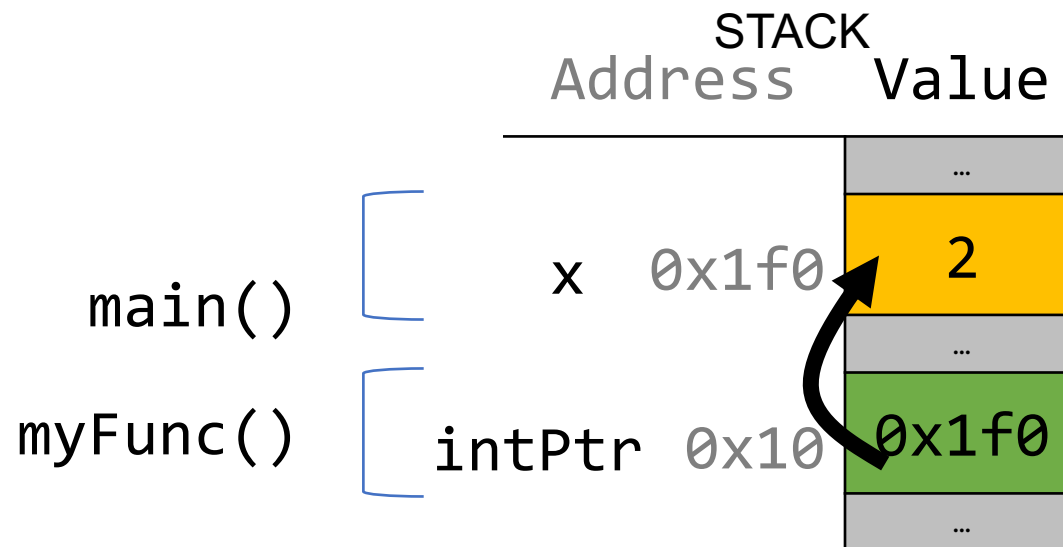
Why does this matter?

- Understanding this representation is key to efficient string manipulation.
- This is how strings are represented in both low- and high-level languages!
 - C++: <https://www.quora.com/How-does-C++-implement-a-string>
 - Python: <https://www.laurentluce.com/posts/python-string-objects-implementation/>

Pointers, Stack and Heap

Key Question: *How can we effectively manage all types of memory in our programs?*

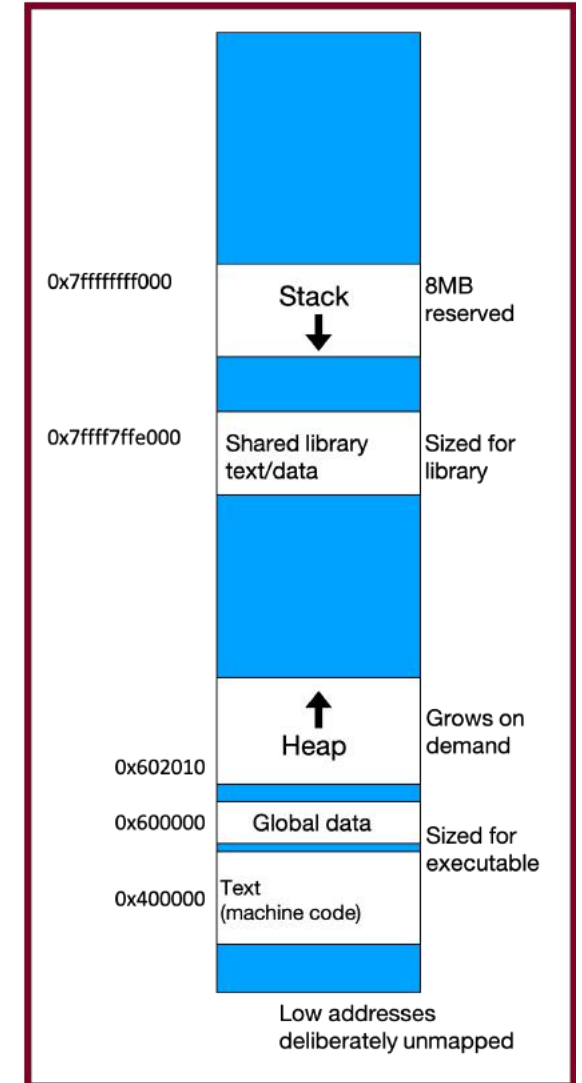
- Arrays let us store ordered lists of information.
- Pointers let us pass addresses of data instead of the data itself.
- We can use the stack, which cleans up memory for us, or the heap, which we must manually manage.



Stack And Heap

Why does this matter?

- The stack and heap allow for two ways to store data in our programs, each with their own tradeoffs, and it's crucial to understand the nuances of managing memory in any program you write!
- Pointers let us pass around references to data efficiently



Generics

Key Question: *How can we use our knowledge of memory and data representation to write code that works with any data type?*

- We can use `void *` to circumvent the type system, `memcpy`, etc. to copy generic data, and function pointers to pass logic around.

Why does this matter?

- Working with any data type lets us write more generic, reusable code.
- Using generics helps us better understand the type system in C and other languages, and where it can help and hinder our program.

Assembly Language

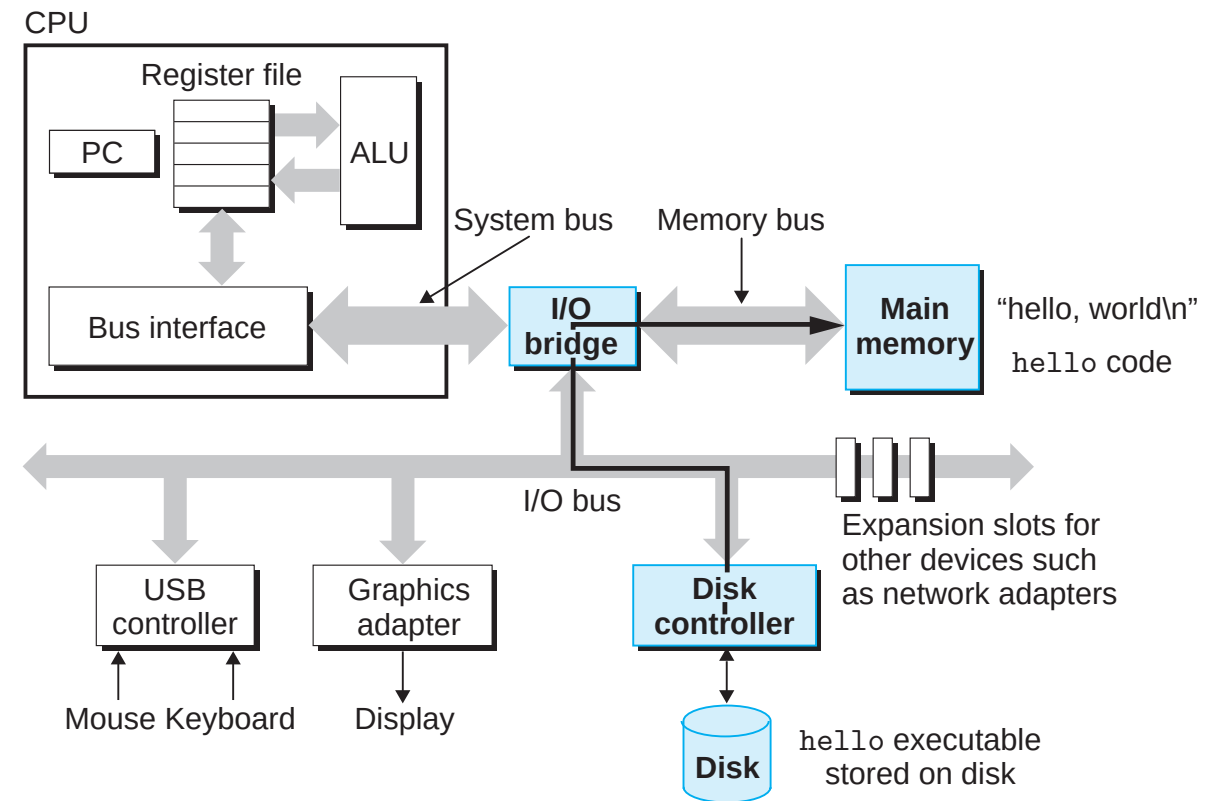
Key Question: *How does a computer interpret and execute C programs?*

- GCC compiles our code into *machine code instructions* executable by our processor.
- Our processor uses registers and instructions like **mov** to manipulate data.

Assembly Language

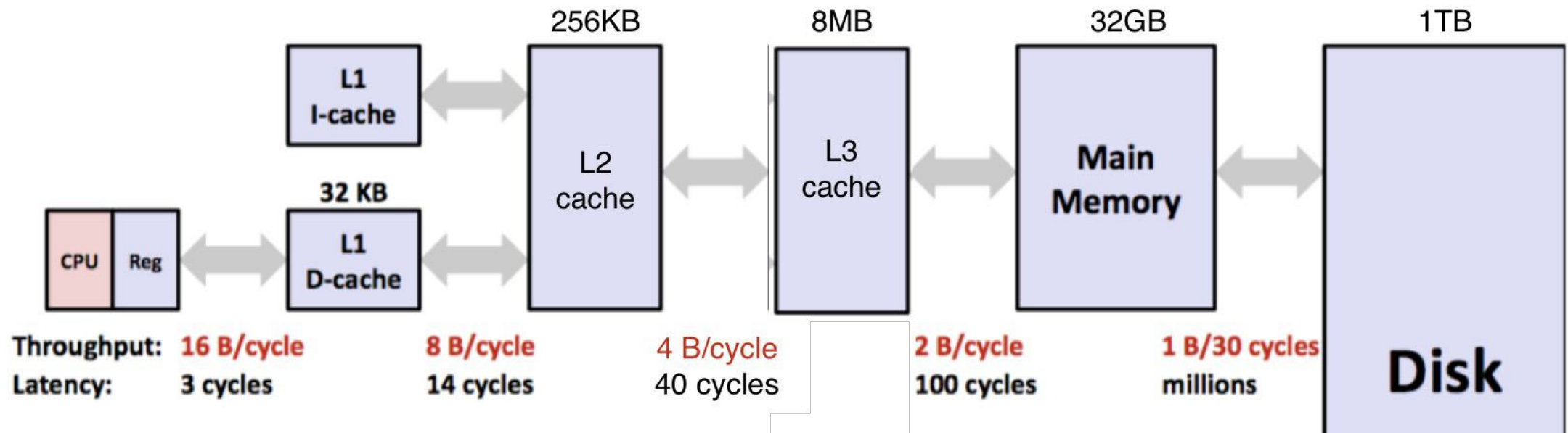
Why does this matter?

- We write C code because it is higher level and transferrable across machines. But it is not the representation executed by the computer!
- Understanding how programs are compiled and executed, as well as computer architecture, is key to writing performant programs (e.g. fewer lines of code is not necessarily better).
- We can reverse engineer and exploit programs at the assembly level!



Caching

- Processor speed is not the only bottleneck in program performance – memory access is perhaps even more of a bottleneck!
- Memory exists in levels and goes from *really fast* (registers) to *really slow* (disk).
- As data is more frequently used, it ends up in faster and faster memory.



Caching

Why does this matter?

Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer.

- **Temporal locality**

- Repeat access to the same data tends to be co-located in TIME
- Intuitively: things I have used recently, I am likely to use again soon

- **Spatial locality**

- Related data tends to be co-located in SPACE
- Intuitively: data that is near a used item is more likely to also be accessed

Linking

- Linking is a technique that allows programs to be constructed from multiple object files.
- Linking can happen at different times in a program's lifetime:
 - Compile time (when a program is compiled)
 - Load time (when a program is loaded into memory)
 - Run time (while a program is executing)

Why does this matter?

- Understanding linking can help you avoid nasty errors and make you a better programmer.

COMP201 Learning Goals

The goals for COMP201 are for students to gain **mastery** of

- writing C programs with complex use of memory and pointers
- an accurate model of the address space and compile/runtime behavior of C programs

to achieve **competence** in

- translating C to/from assembly
- writing programs that respect the limitations of computer arithmetic
- finding bottlenecks and improving runtime performance
- working effectively in a Unix development environment

and have **exposure** to

- a working understanding of the basics of cache memories



Lecture Plan

- Recap: Where We've Been
- COMP201 Tools and Techniques
- What's Next?
- Q&A

Tools and Techniques

- Unix and the command line
- Coding Style
- Debugging (GDB)
- Memory Checking (Valgrind)
- Writing Cache-friendly Code (Callgrind)

Unix And The Command Line

Unix and command line tools are extremely popular tools outside of COMP201 for:

- Running programs (web servers, python programs, remote programs...)
- Accessing remote servers (Amazon Web Services, Microsoft Azure, Heroku...)
- Programming embedded devices (Raspberry Pi, etc.)

Our goal for COMP201 was to help you become proficient in navigating Unix!

Debugging (GDB)

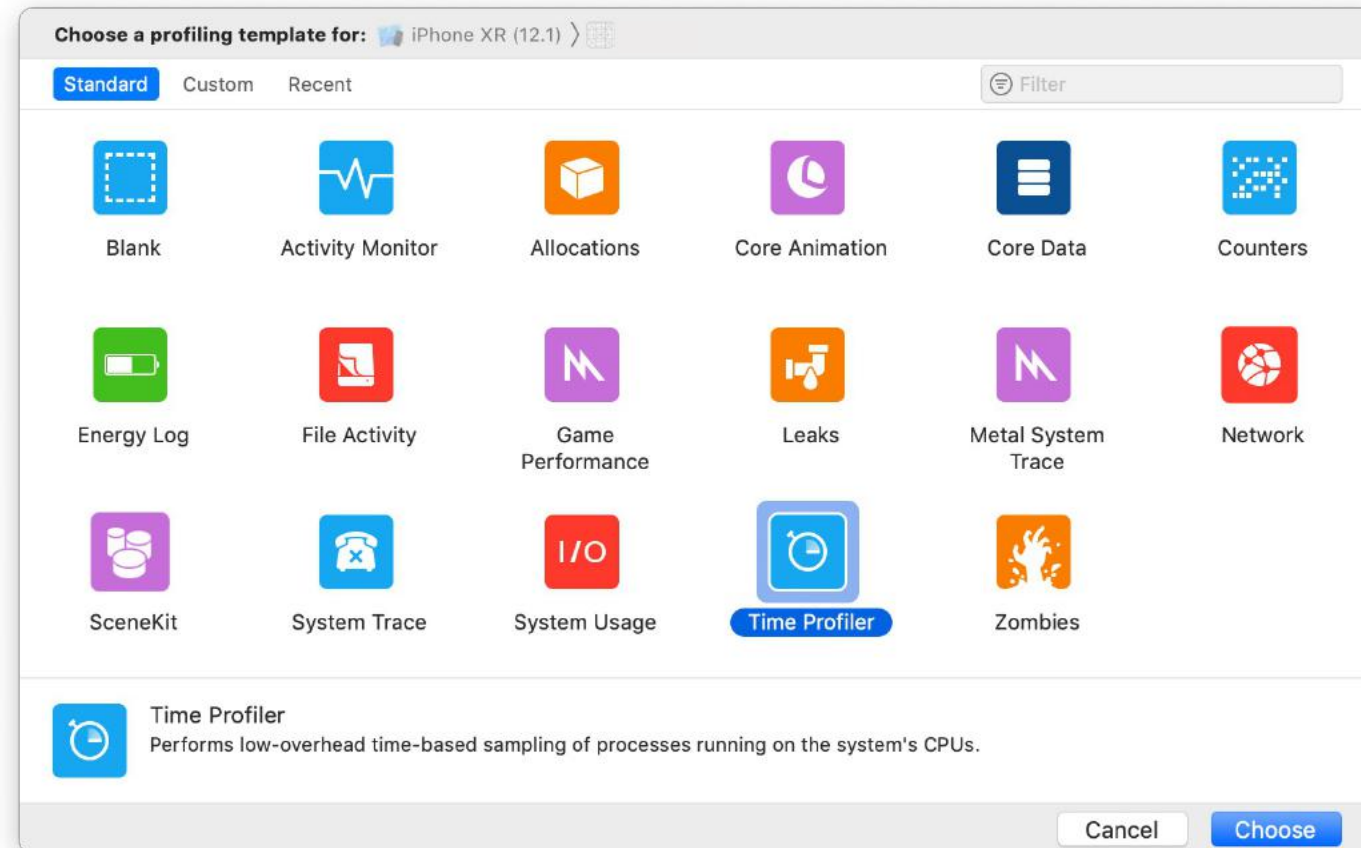
- Debugging is a crucial skill for any computer scientist
- Our goal for COMP201 was to help you become a better debugger
 - narrow in on bugs
 - diagnose the issue
 - implement a fix
- Practically every project you work on will have debugging facilities
 - Python: "PDB"
 - IDEs: built-in debuggers (e.g. QT Creator, Eclipse)
 - Web development: in-browser debugger

Memory Checking and Profiling

- Memory checking and profiling are crucial for any computer scientist to analyze program performance and increase efficiency.
- Many projects you work on will have profiling and memory analysis facilities:
 - Mobile development: integrated tools (XCode Instruments, Android Profiler, etc.)
 - Web development: in-browser tools

Tools

You'll see manifestations of these tools throughout projects you work on. We hope you can use your COMP201 knowledge to take advantage of them!

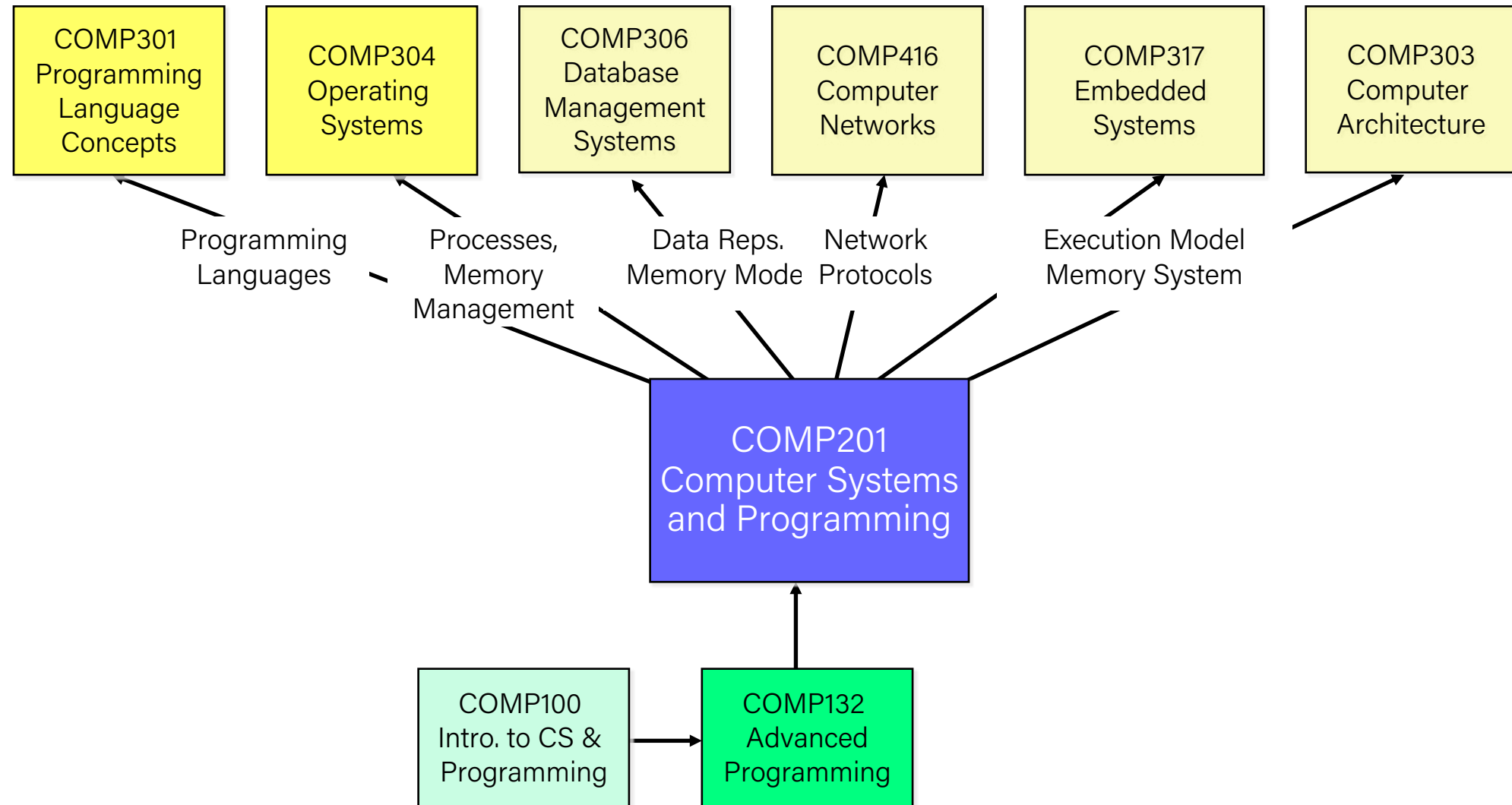


Lecture Plan

- **Recap:** Where We've Been
- Larger Applications
- What's Next?
- Q&A

After COMP201, you are
prepared to take a variety of
classes in various areas.
What are some options?

Role within COMP Curriculum



Courses

How is an operating system implemented? (take COMP304!)

- Threads
- User Programs
- Virtual Memory
- Filesystem

How is a programming language designed? (take COMP301!)

- Lexical analysis
- Parsing
- Semantic Analysis
- Code Generation

Courses

**What are the principles of designing computer hardware
(take COMP303)**

- Computer organization

**How can we write programs that execute on special hardware?
(take COMP317!)**

- Embedded systems

**How can applications communicate over a network?
(take COMP416!)**

- How can we weigh different tradeoffs of network architecture design?
- How can we effectively transmit bits across a network?

Machine Learning

Can we speed up machine learning training with reduced precision computation?

- <https://www.top500.org/news/ibm-takes-aim-at-reduced-precision-for-new-generation-of-ai-chips/>
- <https://devblogs.nvidia.com/mixed-precision-training-deep-neural-networks/>

How can we implement performant machine learning libraries?

- Popular tools such as TensorFlow and PyTorch are implemented using C!
- <https://pytorch.org/blog/a-tour-of-pytorch-internals-1/>
- <https://www.tensorflow.org/guide/extend/architecture>

Web Development

How can we efficiently translate Javascript code to machine code?

- The Chrome V8 JavaScript engine converts Javascript into machine code for computers to execute: <https://medium.freecodecamp.org/understanding-the-core-of-nodejs-the-powerful-chrome-v8-engine-79e7eb8af964>
- The popular Node.js web server tool is built on Chrome V8

How can we compile programs into an efficient binary instruction format that runs in a web browser?

- WebAssembly is an emerging standard instruction format that runs in browsers: <https://webassembly.org>
- You can compile C/C++/other languages into WebAssembly for [web execution](#)

Programming Languages / Runtimes

How can programming languages and runtimes efficiently manage memory?

- Manual memory management (C/C++)
- Reference Counting (Swift)
- Garbage Collection (Java)

How can we design programming languages to reduce the potential for programmer error?

- Haskell/Swift "Optionals"

How can we design portable programming languages?

- Java Bytecode: https://en.wikipedia.org/wiki/Java_bytecode

Theory

How can compilers output efficient machine code instructions for programs?

- Languages can be represented as regular expressions and context-free grammars
- We can model programs as control-flow graphs for additional optimization

Security

How can we find / fix vulnerabilities at various levels in our programs?

- Understand machine-level representation and data manipulation
- Understand how a computer executes programs
- macOS High Sierra Root Login Bug: https://objective-see.com/blog/blog_0x24.html

Lecture Plan

- **Recap:** Where We've Been
- Larger Applications
- What's Next?
- Q&A

Course Evaluations

- I hope you can take the time to fill out the end-semester COMP201 course evaluation.
- I sincerely appreciate any feedback you have about the course and read every piece of feedback we receive.
- I am always looking for ways to improve!

Q&A

